

Backend Documentation: AI-Based Web Exam Simulation

1. Overview

The backend of the web exam simulation is a RESTful API built with Django and the Django REST Framework (DRF). It serves as the central engine managing business logic, ranging from exam creation and student enrollment to advanced AI-driven workflows like automated question generation and secure identity verification.

2. Technology Stack

- **Core Framework:** Django & Django REST Framework.
- **Database:** Designed to integrate with MySQL, though SQLite artifacts (db.sqlite3) are ignored in version control for local development.
- **Computer Vision (CV):** DeepFace (utilizing the VGG-Face model and RetinaFace detector) and OpenCV for facial recognition and identity verification.
- **Natural Language Processing (NLP):** LangChain, Chroma (Vector Database), HuggingFace (Sentence Transformers), and the Groq API for the Retrieval-Augmented Generation (RAG) pipeline.
- **PDF Processing:** pypdf for extracting raw text from uploaded lecture materials.
- **Containerization:** Docker and Docker Compose.

3. Core Modules & Architecture

A. Authentication & Authorization

- **Token-Based Auth:** Uses DRF's TokenAuthentication.
- **Role-Based Access:** The system differentiates between Professors (is_staff=True) and Students. Only staff members can create exams, upload lecture materials, and manage enrollments.

B. Exam & Enrollment Management

- **Exam Models:** Relies on Exam, ExamEnrollment, and ExamParticipation models.
- **Access Control:** Students can only fetch exam questions and join an exam if their specific matriculation number has been whitelisted in the ExamEnrollment records by a professor.
- **Exam Storage:** Finalized exams are stored locally as JSON files within the uploads/exams/ directory.

C. Identity Verification Engine

- **Process:** The verify_identity module processes two image uploads via multipart/form-data: a live webcam capture and an ID card photo.
- **Analysis:** It extracts faces from both images and calculates the facial distance. If the distance is below the strict MATCH_THRESHOLD of 0.80, the identity is confirmed.
- **Fallback:** If the automated verification fails, students can trigger a help-request to log an entry in manual_checks.jsonl for a manual review by the professor.

D. Advanced RAG System (advanced_rag.py)

The automated question generation pipeline utilizes a multi-step Retrieval-Augmented Generation approach:

1. **Ingestion & Chunking:** Uploaded PDFs are parsed and split into text chunks (size: 1000 characters, overlap: 50 characters).
2. **Embedding & Indexing:** Chunks are embedded using sentence-transformers/all-MiniLM-L6-v2 and stored in a local Chroma vector database inside the db_chroma_vorlesung directory.
3. **Retrieval & Re-Ranking:** The system retrieves top chunks based on similarity search and then re-ranks them using a Cross-Encoder (ms-marco-MiniLM-L-6-v2) to ensure high contextual relevance.
4. **LLM Generation:** The highly relevant context is passed to the Groq API (using the openai/gpt-oss-20b model) with a rigid prompt instructing the LLM to act as a university professor and format multiple-choice questions in JSON.

4. Deployment & Setup

The backend is fully containerized and managed via Docker Compose.

- **Service Configuration:** The backend service is built from the ./AiExam directory and mapped to host port 8000.
- **Environment Variables:** *
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1,backend,0.0.0.0
 - GROQ_API_KEY: Must be supplied in the environment for the RAG pipeline to authenticate with the LLM provider.
- **Ignored Directories:** Virtual environments (venv/), Python bytecodes (__pycache__), local databases, and IDE configurations are explicitly excluded from the Docker build and Git tracking.